CSS White-Space

Complete Guide: Controlling Whitespace and Text Wrapping

Contents

1 Introduction

The CSS `white-space` property controls how whitespace (spaces, tabs, newlines) is handled in text. This fundamental property affects text wrapping, line breaking, and formatting preservation.

1.1 Whitespace Challenges

Without proper whitespace control:

- Multiple spaces collapse into one
- Line breaks are ignored
- Text wraps unexpectedly
- Formatted text becomes unformatted
- Code displays incorrectly
- Pre-formatted content loses structure

1.2 Importance of white-space

The `white-space` property is critical for:

- Code block formatting
- Poetry and pre-formatted text
- Preserving intentional spacing
- Controlling text wrapping
- Maintaining visual layout
- Handling special content types

2 white-space Property Values

2.1 Complete Value Overview

Value	Spaces	Tabs	Newlines	Wraps
`normal`	Collapse	Collapse	Collapse	Yes
`nowrap`	Collapse	Collapse	Collapse	No
`pre`	Preserve	Preserve	Preserve	No
`pre-wrap`	Preserve	Preserve	Preserve	Yes
`pre-line`	Collapse	Collapse	Preserve	Yes

3 white-space: normal

The default value. Whitespace is collapsed and text wraps normally.

3.1 Behavior

- Multiple spaces become one space
- Tabs become one space
- Newlines become one space
- Text wraps at container edge
- Soft hyphens work

3.2 Examples

```
/* Default behavior */
p {
  white-space: normal;
}

/* HTML with extra whitespace */
<p>This   has     multiple spaces
and newlines  but they
collapse  to  single  spaces.</p>

/* Renders as: This has multiple spaces and
   newlines but they collapse to single spaces. */
```

4 white-space: nowrap

Prevents text wrapping while collapsing whitespace.

4.1 Behavior

- Whitespace collapses (like normal)
- Text does NOT wrap
- Content may overflow container
- Forces single line layout
- Often paired with text-overflow

4.2 Examples

```
/* Single line, no wrapping */
.no-wrap {
  white-space: nowrap;
}

/* Truncate with ellipsis */
.truncated {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Header that stays on one line */
h1 {
  white-space: nowrap;
}

/* Table cell */
td {
  white-space: nowrap;
}

/* Username */
.username {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

5 white-space: pre

Preserves all whitespace exactly as typed. Like HTML `<pre>` tag.

5.1 Behavior

- All spaces preserved
- All tabs preserved
- All newlines preserved
- Text does NOT wrap
- Content may overflow container
- Font is typically monospace

5.2 Examples

```
/* Pre-formatted code */
code {
  white-space: pre;
  font-family: 'Courier New', monospace;
}

/* ASCII art */
.ascii-art {
  white-space: pre;
  font-family: monospace;
}

/* Preserving exact spacing */
<code>Function name:   calculate
  Parameter count: 3
  Return type:     boolean</code>

<style>
code {
  white-space: pre;
}
</style>

/* Poetry with preserved formatting */
.poem {
  white-space: pre;
  font-family: Georgia, serif;
}
```

6 white-space: pre-wrap

Preserves whitespace AND allows text wrapping. Best for pre-formatted content that needs to fit containers.

6.1 Behavior

- All spaces preserved
- All tabs preserved
- All newlines preserved
- Text DOES wrap at edge
- Combines benefits of pre and normal
- Most flexible for formatted content

6.2 Examples

```
/* Code that wraps */
code {
  white-space: pre-wrap;
  font-family: monospace;
}

/* Formatted text that wraps */
.formatted {
  white-space: pre-wrap;
}

/* User-provided code */
<pre>
function processData(input) {
  const result = input
    .filter(item => item.active)
    .map(item => item.value);
  return result;
}
</pre>

<style>
pre {
  white-space: pre-wrap;
  max-width: 600px;
}
</style>

/* Chat with preserved formatting */
.chat-message {
  white-space: pre-wrap;
  word-wrap: break-word;
}
```

7 white-space: pre-line

Collapses spaces/tabs but preserves newlines. Hybrid approach.

7.1 Behavior

- Multiple spaces collapse to one
- Tabs collapse to one space
- Newlines ARE preserved
- Text DOES wrap normally
- Good for structured text

7.2 Examples

```
/* Preserving line breaks but not spaces */
.addresses {
  white-space: pre-line;
}

/* HTML example */
<div class="addresses">
John Smith
123 Main Street
New York, NY 10001

Jane Doe
456 Oak Avenue
Los Angeles, CA 90001
</div>

<style>
.addresses {
  white-space: pre-line;
}
</style>

/* Poetry with line breaks */
.poem {
  white-space: pre-line;
  font-family: Georgia, serif;
}

/* Structured list */
.structured-list {
  white-space: pre-line;
  font-family: monospace;
}
```

8 Real-World Use Cases

8.1 Use Case 1: Code Blocks

```
pre {
  background-color: #f4f4f4;
  border: 1px solid #ddd;
  border-radius: 4px;
  padding: 15px;
  overflow-x: auto;
  white-space: pre-wrap;
  font-family: 'Courier New', monospace;
  font-size: 14px;
  line-height: 1.4;
}

code {
  white-space: pre-wrap;
}

/* Inline code */
code.inline {
  white-space: nowrap;
  padding: 2px 4px;
  background-color: #f0f0f0;
  border-radius: 2px;
}
```

8.2 Use Case 2: Truncated Text

```
/* Table cells */
td {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: 200px;
}

/* Breadcrumb navigation */
.breadcrumb {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Single line title */
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

```
/* File path */
.filepath {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  font-family: monospace;
}
```

8.3 Use Case 3: Formatted Address

```
.address {
  white-space: pre-line;
  font-style: normal;
  line-height: 1.6;
}

/* HTML */
<div class="address">
John Doe
123 Main Street
New York, NY 10001
USA
</div>

/* Alternative: preserving spaces too */
.address-formatted {
  white-space: pre-wrap;
  font-family: monospace;
}
```

8.4 Use Case 4: ASCII Art

```
.ascii-art {
  white-space: pre;
  font-family: monospace;
  font-size: 12px;
  line-height: 1;
}

/* HTML */
<pre class="ascii-art">
    *
   / \
  /   \
 /_____\
  | | |
 _| | |_
</pre>

/* In CSS */
.ascii {
```

```
  white-space: pre;
  font-family: monospace;
  overflow-x: auto;
}
```

8.5 Use Case 5: User-Generated Content

```
.user-content {
  white-space: pre-wrap;
  word-wrap: break-word;
  overflow-wrap: break-word;
}

/* Chat message */
.chat-message {
  white-space: pre-wrap;
  word-break: break-word;
  max-width: 400px;
}

/* Comment */
.comment {
  white-space: pre-wrap;
  word-break: break-word;
}

/* Forum post */
.forum-post {
  white-space: pre-line;
  word-wrap: break-word;
}
```

9 Combining white-space with Other Properties

9.1 white-space and word-break

```
/* Preserve formatting and break long words */
pre {
  white-space: pre-wrap;
  word-break: break-word;
}

/* Preserve formatting, don't break words */
pre {
  white-space: pre-wrap;
  word-break: normal;
  overflow-x: auto;  /* Horizontal scroll */
}

/* Preserve lines, handle long words */
.formatted {
```

```
  white-space: pre-line;
  word-break: break-word;
}
```

9.2 white-space and overflow

```
/* Hide overflow, truncate */
.truncated {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Show scrollbar */
code {
  white-space: pre;
  overflow-x: auto;
  overflow-y: hidden;
}

/* Pre-formatted with wrapping */
.pre-wrap {
  white-space: pre-wrap;
  max-width: 600px;
}
```

9.3 white-space and text-align

```
/* Pre-formatted, centered */
.center-code {
  white-space: pre;
  text-align: center;
  font-family: monospace;
}

/* Monospace text, right-aligned */
.right-code {
  white-space: pre;
  text-align: right;
  font-family: monospace;
}

/* Justified code (unusual) */
pre {
  white-space: pre-wrap;
  text-align: justify;
}
```

10 Browser Behavior

10.1 Browser Compatibility

Value	Chrome	Firefox	Safari	Edge
normal	All	All	All	All
nowrap	All	All	All	All
pre	All	All	All	All
pre-wrap	All	All	All	All
pre-line	All	All	All	All

10.2 Universal Support

All `white-space` values are universally supported across all modern browsers. No fallbacks or polyfills needed.

11 Common Issues and Solutions

11.1 Issue 1: Code Overflow

Problem:

```
pre {
  white-space: pre;
  /* Long lines overflow */
}
```

Solution:

```
/* Option 1: Horizontal scroll */
pre {
  white-space: pre;
  overflow-x: auto;
}

/* Option 2: Wrap content */
pre {
  white-space: pre-wrap;
  max-width: 600px;
}

/* Option 3: Font size adjustment */
pre {
  white-space: pre;
  font-size: 12px;
  overflow-x: auto;
}
```

11.2 Issue 2: Text Not Wrapping

Problem:

```
p {
  white-space: nowrap;
  /* Text doesn't wrap on mobile */
}
```

Solution:

```
p {
  white-space: normal;  /* Default wrapping */
}

/* Or, conditional based on screen size */
@media (max-width: 768px) {
  .title {
    white-space: normal;
  }
}

@media (min-width: 769px) {
```

```
  .title {
    white-space: nowrap;
  }
}
```

11.3 Issue 3: Spaces Not Preserved

Problem:

```
.formatted {
  white-space: normal;
  /* Intentional spaces collapse */
}
```

Solution:

```
.formatted {
  white-space: pre;  /* Preserve spaces */
}

/* Or use HTML entities */
<p>Space:  preserved</p>

/* Or use CSS */
.spacing {
  letter-spacing: 2px;
  word-spacing: 5px;
}
```

11.4 Issue 4: Line Breaks Not Working

Problem:

```
.text {
  white-space: normal;
  /* Newlines in HTML ignored */
}
```

Solution:

```
.text {
  white-space: pre-line;  /* Preserve line breaks */
}

/* Or use <br> tags */
<p>Line 1<br>Line 2<br>Line 3</p>

/* Or CSS generated content */
.text {
  white-space: pre-wrap;
}
```

12 Responsive white-space

Adjust whitespace handling for different screens.

12.1 Examples

```
/* Desktop: truncate */
@media (min-width: 1024px) {
  .title {
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
  }
}

/* Tablet: wrap */
@media (max-width: 1023px) and (min-width: 768px) {
  .title {
    white-space: normal;
  }
}

/* Mobile: wrap with normal spacing */
@media (max-width: 767px) {
  .title {
    white-space: normal;
  }

  code {
    white-space: pre-wrap;
    word-break: break-word;
  }
}
```

13 Performance Considerations

13.1 Performance Impact

- **white-space: normal** - No performance cost
- **white-space: nowrap** - Minimal impact
- **white-space: pre** - No performance cost
- **white-space: pre-wrap** - Minor impact (text layout)
- **white-space: pre-line** - Minimal impact

13.2 Optimization Tips

```
/* Use pre-wrap only where needed */
code {
  white-space: pre-wrap;  /* Only on code */
}

p {
  white-space: normal;  /* Default elsewhere */
}

/* Combine with container constraints */
pre {
  white-space: pre-wrap;
  max-width: 600px;  /* Limit reflow */
}

/* Use nowrap for performance in lists */
.item-title {
  white-space: nowrap;  /* Prevents layout shift */
}
```

14 Accessibility Considerations

14.1 Best Practices

- Preserve semantic meaning with whitespace
- Use pre-formatted text appropriately
- Ensure code samples are readable
- Test with screen readers
- Maintain sufficient contrast

14.2 Accessible Examples

```
/* Semantic code block */
<pre>
  <code>
    function example() {
      return true;
    }
  </code>
</pre>

<style>
pre {
  background-color: #f4f4f4;
  padding: 15px;
  border-radius: 4px;
  white-space: pre-wrap;
  overflow-x: auto;
```

```
}

code {
  font-family: monospace;
  font-size: 14px;
}
</style>

/* Alternative for complex code */
<pre>
<code>
  Your code here
</code>
</pre>

/* Aria label for clarity */
<pre aria-label="Example JavaScript function">
  <code>function demo() { }</code>
</pre>
```

15 Common Mistakes

15.1 Mistake 1: Using pre for All Text

Bad:

```
body {
  white-space: pre;
  /* Everything becomes pre-formatted */
}
```

Good:

```
body {
  white-space: normal;  /* Default */
}

code {
  white-space: pre;  /* Only code blocks */
}

pre {
  white-space: pre-wrap;
}
```

15.2 Mistake 2: Forgetting Overflow

Bad:

```
code {
  white-space: pre;
  /* Content overflows without scroll */
}
```

Good:

```
code {
  white-space: pre;
  overflow-x: auto;  /* Add scroll */
}

/* Or wrap */
code {
  white-space: pre-wrap;
}
```

15.3 Mistake 3: Mixing Whitespace Handling

Bad:

```
.container {
  white-space: nowrap;
  /* Child elements forced nowrap */
}

p {
  white-space: normal;  /* Doesn't override parent */
}
```

Good:

```
.title {
  white-space: nowrap;  /* Intentional */
}

p {
  white-space: normal;  /* Separate rule */
}
```

16 Advanced Techniques

16.1 Pre-formatted with Highlighting

```
/* Formatted code with syntax highlighting */
<pre class="highlight">
  <code>
    <span class="keyword">function</span>
    <span class="function-name">example</span>() {
      <span class="keyword">return</span>
      <span class="value">true</span>;
    }
  </code>
</pre>

<style>
pre {
  white-space: pre-wrap;
```

```
  background-color: #1e1e1e;
  color: #d4d4d4;
  padding: 15px;
  border-radius: 4px;
  overflow-x: auto;
}

.keyword { color: #569cd6; }
.function-name { color: #dcdcaa; }
.value { color: #ce9178; }
</style>
```

16.2 Responsive Code Blocks

```
/* Desktop: horizontal scroll */
@media (min-width: 1024px) {
  pre {
    white-space: pre;
    overflow-x: auto;
  }
}

/* Tablet: balance scroll and wrap */
@media (max-width: 1023px) and (min-width: 768px) {
  pre {
    white-space: pre-wrap;
    max-width: 100%;
  }
}

/* Mobile: wrap aggressively */
@media (max-width: 767px) {
  pre {
    white-space: pre-wrap;
    font-size: 12px;
  }
}
```

17 Summary: Key Takeaways

17.1 white-space Values Quick Reference

Value	Use	Common For
`normal`	Default	Regular text
`nowrap`	Single line	Titles, truncation
`pre`	Fixed format, no wrap	Code, ASCII art
`pre-wrap`	Fixed format + wrap	Code blocks
`pre-line`	Preserve breaks	Addresses, lists

17.2 Best Practices

1. Use `normal` as default
2. Use `pre-wrap` for code blocks
3. Use `nowrap` for truncation (with text-overflow)
4. Combine with overflow properties
5. Test on different screen sizes
6. Preserve semantic HTML meaning
7. Consider accessibility
8. Use monospace fonts with pre-formatted text
9. Provide scrollbars for long content
10. Document unusual whitespace handling

17.3 Quick Reference

```
/* Regular text (default) */
p {
  white-space: normal;
}

/* Single line with ellipsis */
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Code block with wrapping */
pre {
  white-space: pre-wrap;
  font-family: monospace;
  overflow-x: auto;
}

/* Pre-formatted, no wrap */
code {
  white-space: pre;
  font-family: monospace;
  overflow-x: auto;
}

/* Formatted text with line breaks */
.address {
  white-space: pre-line;
}
```

```
/* User content */
.user-content {
  white-space: pre-wrap;
  word-break: break-word;
}
```

18 Conclusion

The CSS `white-space` property is fundamental to text formatting on the web. Understanding how to control whitespace preservation, line breaking, and text wrapping is essential for handling diverse content types effectively.

Key principles to remember:

1. `normal` collapses whitespace and wraps text
2. `nowrap` prevents wrapping but collapses whitespace
3. `pre` preserves all whitespace but doesn't wrap
4. `pre-wrap` preserves whitespace AND wraps
5. `pre-line` preserves newlines but not spaces
6. Combine with other properties for full control
7. Always test on different screen sizes
8. Use monospace fonts with pre-formatted text
9. Consider accessibility and semantic HTML
10. Provide appropriate overflow handling

With mastery of the `white-space` property, you'll handle text formatting challenges effectively, from code blocks to user-generated content, ensuring your web applications display content exactly as intended.

Happy coding!